

ACADEMIC SUBMISSION

from:



LEARNWELL INSTITUTE

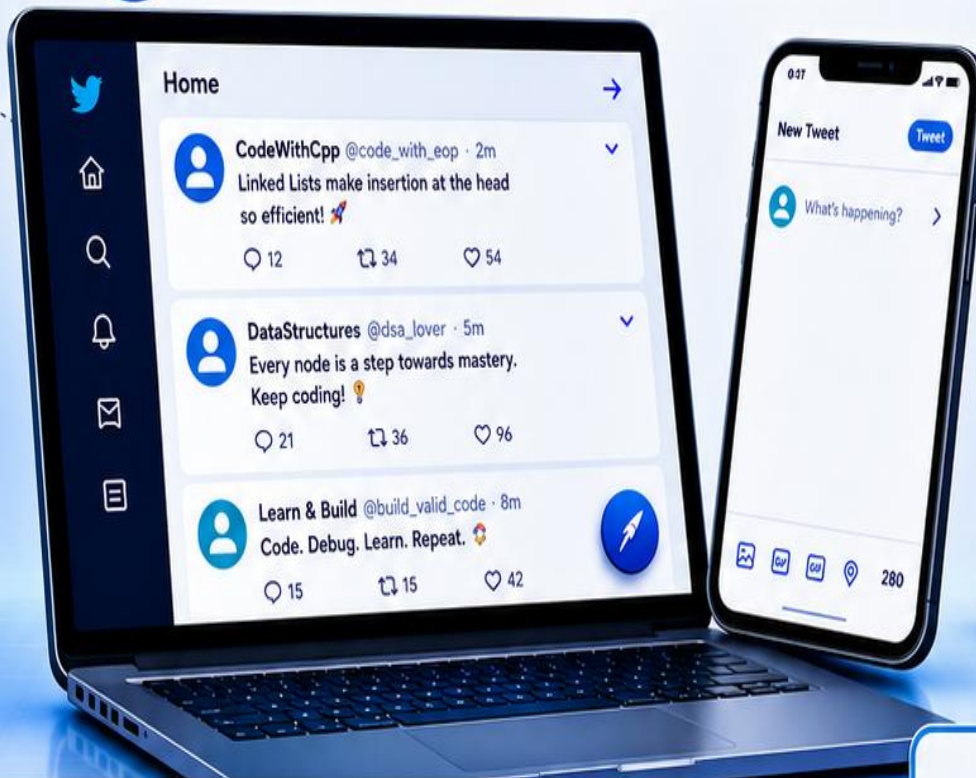
PROJECT REPORT

ON

TWEET MANAGEMENT SYSTEM



A TWITTER-STYLE LINKED LIST ENGINE



HEAD → TWEET 1001 → TWEET 1002 → TWEET 1003 → TWEET 1004 NULL



INSERT
TWEET



DELETE
TWEET



SEARCH
TWEET



TRAVERSE
TIMELINE



REVERSE
LIST



COUNT
NODES



GROUP MEMBERS



Rohit Sharma



Kartik Tak



Aryan



STUDENT NAME : Dilkhush Kumar



COURSE : BCA



SEMESTER : 2nd Semester



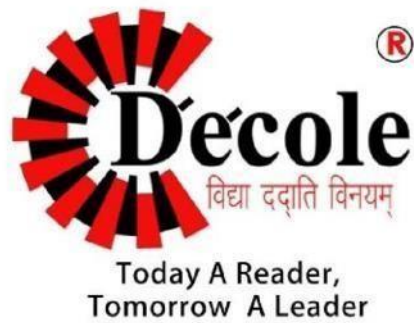
SUBJECT : C++ Programming &
Data Structure



SESSION : 2025-26



COLLEGE : Dezyne École College



Dezyne École College
www.dezyneecole.com

Title of project
TWEET MANAGEMENT SYSTEM

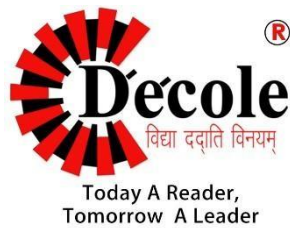
SUBJECT NAME
DATA STRUCTURE & C++

Submitted by
Dilkhush kumar
Rohit sharma
Aryan deep singh
Kartik tak

Department – BCA
Semester – second
Session - 2025-26

Project Report of
TWEET MANAGEMENT
SYSTEM

Submitted to
Dezyne École College
Towards the Partial
Fulfilment of **BCA Second**



Semester

By
Dilkhush Kumar

2025-2026

Acknowledgement

We take this opportunity to express our sincere gratitude to Dezyne École College for providing us with the knowledge, resources, and guidance essential to the successful completion of this project.

Our deepest thanks to our respected Principal, Dr. Vinita Mathur, for her visionary leadership and continuous encouragement. Her commitment to academic excellence and student growth has been a constant source of motivation and pride.

We are equally thankful to all our faculty members for their consistent support, expert advice, and valuable insights that shaped our learning and helped us achieve this milestone.

Lastly, we extend heartfelt thanks to everyone who directly or indirectly contributed to the success of this project.

We are proud to be part of Dezyne École College, an institution that truly nurtures professionalism, innovation, and purpose.

With sincere appreciation,

Dilkhush Kumar
Rohit sharma
Aryan deep singh
Kartik tak

Department of BCA
Dezyne École College, Ajmer

TWITTER TIMELINE

A Linked List Project in C++

Singly & Doubly Linked List Implementation

Data Structures with C++
Project Documentation

Contents

Part 1 — Introduction: Linked Lists & Twitter	2
What is a Linked List?	2
Why Twitter? — The Real-World Context	2
Why not an Array?	2
Singly vs Doubly — At a Glance	2
Part 2 — C++ Implementation	4
Driver Module — MainProjectLinkedList.cpp	4
Common Design Choices	5
Part 3 — Singly Linked List: All Operations	6
Node Structure	6
List Class — LinkedList	6
3.1 Insert at Beginning — $O(1)$	7
3.2 Insert at End — $O(1)$	7
3.3 Insert at Given Position — $O(k)$	8
3.4 Delete from Beginning — $O(1)$	9
3.5 Delete from End — $O(n)$	10
3.6 Delete from Given Position — $O(k)$	10
3.7 Traversal (showNode) — $O(n)$	11
3.8 Search by User Name — $O(n)$	12
3.9 Count Nodes — $O(1)$	13
3.10 Menu Driver — MainMenu()	13
Part 4 — Doubly Linked List: All Operations	14
Node Structure (NODED)	14
List Class — DLinkedList	14
4.1 Insert at Beginning — $O(1)$	15
4.2 Insert at End — $O(1)$	15
4.3 Insert at Given Position — $O(k)$	16
4.4 Delete from Beginning — $O(1)$	17
4.5 Delete from End — $O(1)$ ★ Key Doubly Advantage	17
4.6 Delete from Given Position — $O(k)$	18
4.7 Forward & Backward Traversal — $O(n)$ each	18
4.8 Search by User Name — $O(n)$	19
Part 5 — Complete Complexity Comparison	21
When to Use Which?	21
Part 6 — Conclusion	22

Part 1 — Introduction: Linked Lists & Twitter

What is a Linked List?

A Linked List is a linear data structure where elements (called nodes) are NOT stored in contiguous memory locations. Each node holds its data and one or two pointers — memory addresses — that link it to neighbouring nodes. The nodes can live anywhere in RAM; the pointers stitch them into a logical sequence.

- Singly Linked List — each node has DATA + 1 pointer (NEXT). Can only traverse forward.
- Doubly Linked List — each node has DATA + 2 pointers (PREV + NEXT). Can traverse in both directions.
- Head / Tail — pointers maintained by the list class so we can locate the first and last node in $O(1)$.

Why Twitter? — The Real-World Context

Twitter's home timeline shows tweets in reverse-chronological order — newest at the top, oldest at the bottom. Three operations dominate this system:

- **Insert at head:** A new tweet always goes at the very top — must be instant — $O(1)$.
- **Delete from any position:** Content moderation removes any tweet immediately.
- **Traverse from head to tail:** Displaying the timeline scrolls from newest to oldest — $O(n)$.

These three needs map perfectly to a Linked List. In our project, every node represents one tweet — each carrying a Tweet ID (TID), the user name (UNAME) and the tweet text (TWEET).

Why not an Array?

Operation	Array	Linked List	Twitter's need
Insert at front	$O(n)$ — shift all elements	$O(1)$ — rewire 2 pointers	New tweet on top, instant
Delete any element	$O(n)$ — shift remaining	$O(1)$ — rewire neighbours	Remove any tweet instantly
Dynamic size	Reallocation needed	Just add a node	Tweet count is unpredictable
Access by index	$O(1)$	$O(n)$	Not needed for sequential feed

Singly vs Doubly — At a Glance

Property	Singly Linked List	Doubly Linked List
Pointers per node	1 (NEXT only)	2 (PREV + NEXT)
Traversal direction	Forward only	Forward & Backward
Delete tail	$O(n)$ — scan to 2nd-last	$O(1)$ — use END->PREV
Delete any node (with ptr)	$O(n)$ — scan for predecessor	$O(1)$ — both neighbours known
Extra memory	Less ($1 \text{ ptr} \times n$)	More ($2 \text{ ptr} \times n$)
Typical use	Stacks, hash chains, simple queues	Timelines, browsers, LRU cache

Part 2 — C++ Implementation

The project is split into three C++ source files. Each file owns one responsibility, and they are stitched together by a small driver class.

File	Responsibility
LinkedList.cpp	Singly Linked List class (NODE + LinkedList) with all menu-driven operations.
doublyll.cpp	Doubly Linked List class (NODED + DLinkedList) with forward/backward operations.
MainProjectLinkedList.cpp	Driver — contains main() and the MainLinkedList class that lets the user pick Singly or Doubly.

Driver Module — MainProjectLinkedList.cpp

The driver creates one Singly list object and one Doubly list object. It shows a top-level menu and forwards control to whichever list the user chooses. Every sub-menu can return back here.

File: MainProjectLinkedList.cpp

```
#include <iostream>
#include <conio.h>
#include "LinkedList.cpp"
#include "doublyll.cpp"

class MainLinkedList
{
public:
    LinkedList l;    // singly linked list object
    DLinkedList dl;  // doubly linked list object

    void MainLinkedListModule()
    {
        system("cls");
        cout << "----- Welcome In Link List Project -----\\n\\n";
        cout << "1. Singly Link List \\n\\n";
        cout << "2. Doubly Link List \\n\\n";
        cout << "3. Exit \\n\\n";
        cout << "Choose any one of them....";

        char c = getch();
        if (c == '1') l.MainMenu();
        else if (c == '2') dl.MainMenu();
        else if (c == '3') exit(0);
    }
};

int main()
{
```

```
MainLinkedList mll;  
mll.MainLinkedListModule();  
cout << "\n\nThank You for Using My App..\n\n";  
cout << "Design & Coded By -- Tarun Sharma..\n\n";  
}
```

Common Design Choices

- Each list class keeps two pointers — START (head) and END (tail) — so insert-at-end and delete-from-end can run in $O(1)$.
- countNode tracks the size of the list, avoiding an $O(n)$ recount.
- TCNT auto-generates a unique Tweet ID like "TWEET1001", "TWEET1002", ... — mimicking Twitter's monotonically increasing tweet IDs.
- Every operation is wrapped behind a numbered console menu (getch()), and a continueCheck() prompt lets the user repeat, return, or exit.

Part 3 — Singly Linked List: All Operations

Node Structure

Each node stores the tweet's data and a single NEXT pointer that holds the address of the following node. The last node's NEXT is nullptr.

File: *LinkedList.cpp* — class *NODE*

```
class NODE
{
public:
    string ID;
    string UNAME;
    string TWEET;
    NODE *NEXT;
    NODE(string x, string y, string z)
    {
        ID = x;
        UNAME = y;
        TWEET = z;
        NEXT = nullptr;
    }
};
```

Memory layout:

```
START -> [TWEET1003 | @niharika | "Hi Pooja"] -> [TWEET1002 | @pooja | "Hello"] -> [TWEET1001 | @deepak | "Hey"] -> NULL
                                (newest)                                (oldest = END)
```

List Class — Link List

The LinkedList class owns START, END, countNode, and the input buffer (TID/UName/UTweet). The constructor sets START and END to nullptr and starts TCNT at 1001.

File: *LinkedList.cpp* — class *LinkedList*

```
class LinkedList
{
public:
    string TID;
    int TCNT;
    string UName, UTweet;
    NODE *START, *END;
    int countNode;

    LinkedList(){
        TCNT = 1001;
        countNode = 0;
        START = nullptr;
        END = nullptr;
    }
};
```

```

void getData()
{
    TID = "TWEET" + to_string(TCNT);
    cout << "\nEnter TID -- " << TID << endl;
    cout << "Enter User Name -- ";
    getline(cin, UName);
    cout << "Enter Tweet -- ";
    getline(cin, UTweet);
}
};

```

3.1 Insert at Beginning — $O(1)$

Concept: A new tweet always lands on top of the timeline. We allocate a new node, point its NEXT at the current START, and then move START to the new node. If the list was empty, START and END both point at the new node.

Time Complexity: $O(1)$ — only 2 pointer assignments, regardless of list size.

Algorithm / Steps:

1. Read tweet data into a new NODE.
2. If list is empty, set START = END = TEMP.
3. Otherwise, set TEMP->NEXT = START.
4. Move START = TEMP (head now points to the new node).
5. Increment countNode and TCNT.

From insertNode() — opr == 1

```

// opr == 1 --> Insert at Beginning
cout << "\n\nInsert At Beginning .\n\n";
getData();
NODE *TEMP = new NODE(TID, UName, UTweet);

if (TEMP == nullptr) {
    cout << "Insufficient Memory.. \n";
    return 0;
}

if (START == nullptr) {      // empty list
    END = START = TEMP;
}
else {
    TEMP->NEXT = START;      // new node points to old head
    START = TEMP;           // head moves to new node
}
countNode++;
TCNT++;

```

3.2 Insert at End — $O(1)$

Concept: Because we maintain a tail pointer (END), appending a node is constant-time: link END->NEXT to the new node, then advance END.

Time Complexity: $O(1)$ — END pointer avoids the $O(n)$ walk.

Algorithm / Steps:

6. Read tweet data into a new NODE.
7. If list is empty, set START = END = TEMP.
8. Otherwise, set END->NEXT = TEMP (old tail links to new).
9. Move END = TEMP (tail moves forward).
10. Increment countNode and TCNT.

From insertNode() — opr == 2

```
// opr == 2 --> Insert at End
cout << "\n\nInsert At End .\n\n";
getData();
NODE *TEMP = new NODE(TID, UName, UTweet);

if (TEMP == nullptr) {
    cout << "Insufficient Memory..\n";
    return 0;
}

if (START == nullptr) {    // empty list
    END = START = TEMP;
}
else {
    END->NEXT = TEMP;      // old tail links to new node
    END = TEMP;           // tail pointer moves forward
}
countNode++;
TCNT++;
```

3.3 Insert at Given Position — O(k)

Concept: To insert at position k we walk a temporary pointer (TEMP1) to the node BEFORE position k, then perform the standard 2-pointer rewire: new node looks past predecessor, predecessor now points at new node.

Time Complexity: O(k) — walk k-1 steps + O(1) rewire.

Algorithm / Steps:

11. Allocate a new NODE.
12. Walk TEMP1 from START until it reaches the node at position pos-1.
13. Set TEMP->NEXT = TEMP1->NEXT (new node looks past predecessor).
14. Set TEMP1->NEXT = TEMP (predecessor now points to new).
15. If TEMP1 was END, advance END to the newly-inserted node.

From insertNode() — opr == 3

```
// opr == 3 --> Insert at Given Position
cout << "\n\nInsert At Given Position .\n\n";
cout << "Total Nodes -- " << countNode << "\n\n";
int pos;
getData();
cout << "Enter Position Between 0 to " << countNode << " -- ";
cin >> pos;
```

```

NODE *TEMP = new NODE(TID, UName, UTweet);
if (TEMP == nullptr) { cout << "Insufficient Memory..\n"; return 0; }

NODE *TEMP1 = START;
for (int i = 2; i < pos; i++) { // walk to predecessor
    TEMP1 = TEMP1->NEXT;
}

if (START == nullptr) {
    END = START = TEMP;
}
else {
    TEMP->NEXT = TEMP1->NEXT; // new node looks past predecessor
    TEMP1->NEXT = TEMP;      // predecessor now points to new node
    if (TEMP1 == END) {
        END = TEMP1->NEXT; // update tail if inserted at end
    }
}
countNode++;
TCNT++;

```

3.4 Delete from Beginning — $O(1)$

Concept: Save the current head in TEMP, advance START to the next node, then delete TEMP.
 Special-case: if there is only one node, set both START and END to nullptr.

Time Complexity: $O(1)$.

Algorithm / Steps:

16. If START is nullptr — list is empty, abort.
17. If only one node exists, set START = END = nullptr.
18. Else, save TEMP = START and move START = START->NEXT.
19. Decrement countNode and delete TEMP.

From deleteNode() — opr == 1

```

// opr == 1 --> Delete From Beginning
cout << "\n\nDelete From Beginning .\n\n";

if (START == nullptr) {
    cout << "LinkedList is Empty..\n\n";
    return 0;
}
else if (START->NEXT == nullptr) { // only one node
    TEMP = START;
    END = START = nullptr;
}
else {
    TEMP = START;
    START = START->NEXT; // head moves to next node
}
countNode--;

```

```
delete(TEMP);
cout << "\nNode Deleted Successfully\n\n";
```

3.5 Delete from End — $O(n)$

Concept: Even though we have an END pointer, a singly linked list has no way to step BACK from the tail. We must scan from START until $TEMP \rightarrow NEXT == END$, then unlink END.

Time Complexity: $O(n)$ — must walk to the second-last node.

Algorithm / Steps:

20. If START is nullptr — list is empty, abort.
21. If only one node, set START = END = nullptr and delete it.
22. Else, walk TEMP from START while $TEMP \rightarrow NEXT != END$.
23. Save T1 = END, set $TEMP \rightarrow NEXT = nullptr$, move END = TEMP.
24. Delete T1 and decrement countNode.

From deleteNode() — opr == 2

```
// opr == 2 --> Delete From End
cout << "\n\nDelete From End .\n\n";

if (START == nullptr) { cout << "LinkedList is Empty..\n\n"; return 0; }
else if (START->NEXT == nullptr) {
    TEMP = START;
    END = START = nullptr;
}
else {
    TEMP = START;
    while (TEMP->NEXT != END) { // scan to second-last node
        TEMP = TEMP->NEXT;
    }
    NODE *T1 = END;
    TEMP->NEXT = nullptr;
    END = TEMP; // tail is now second-last
    delete(T1);
}
countNode--;
cout << "\nNode Deleted Successfully\n\n";
```

3.6 Delete from Given Position — $O(k)$

Concept: We walk TEMP to the predecessor of the target node and bypass the target. There are two sub-cases: deleting the tail (must update END) and deleting a middle node (just rewire).

Time Complexity: $O(k)$.

Algorithm / Steps:

25. Read pos from the user.
26. If list has only one node and $pos == 1$, clear START and END.
27. Else walk TEMP to position $pos-1$.
28. If $TEMP \rightarrow NEXT == END$, unlink the tail and update END = TEMP.
29. Else save T1 = TEMP->NEXT, set $TEMP \rightarrow NEXT = T1 \rightarrow NEXT$ and delete T1.

From deleteNode() — opr == 3


```

// opr == 3 --> Delete From Given Position
cout << "\n\nDelete From Given Position .\n\n";
if (START == nullptr) { cout << "LinkedList is Empty..\n\n"; return 0; }

int pos;
cout << "Enter Position for Delete (1 - " << countNode << ") - ";
cin >> pos;

if (pos == 1 && START == END) {    // only one node
    TEMP = START;
    START = END = nullptr;
}
else {
    TEMP = START;
    for (int i = 2; i < pos; i++) {    // walk to predecessor
        TEMP = TEMP->NEXT;
    }
    if (TEMP->NEXT == END) {          // deleting tail
        NODE *T1 = END;
        TEMP->NEXT = nullptr;
        END = TEMP;
    }
    else {                            // deleting middle node
        NODE *T1 = TEMP->NEXT;
        TEMP->NEXT = T1->NEXT;
        delete(T1);
    }
}
countNode--;
cout << "\nNode Deleted Successfully\n\n";

```

3.7 Traversal (show Node) — $O(n)$

Concept: Walk from START following NEXT pointers, printing each tweet. This is exactly how Twitter would render a feed top-to-bottom.

Time Complexity: $O(n)$.

Algorithm / Steps:

30. Start TEMP = START.
31. While TEMP != nullptr, print [TID, UNAME, TWEET].
32. Advance TEMP = TEMP->NEXT.
33. Print "Null" to mark the end of the list.

Method showNode()

```

void showNode()
{
    cout << "\n\nLink List --\n\n";
    cout << "** Head -> ";
    NODE *TEMP = START;

    if (START == nullptr) {

```

```

        cout << "null \n\nlist is empty\n";
    }
    else {
        while (TEMP != nullptr) {
            cout << "[" << TEMP->ID << ", "
                << TEMP->UNAME << ", "
                << TEMP->TWEET << "]" -> ";
            TEMP = TEMP->NEXT;
        }
        cout << "Null";
    }
}

```

3.8 Search by User Name — $O(n)$

Concept: There is no index in a linked list, so we visit every node and compare its UNAME with the search term. We count matches so the user knows how many tweets the user has posted.

Time Complexity: $O(n)$.

Algorithm / Steps:

34. Read the user name to search.
35. Walk TEMP from START to nullptr.
36. If TEMP->UNAME equals the search term, print the tweet and increment counter.
37. If counter == 0 after the walk, print "No Tweet Found".

Method searchTweetInNode()

```

int searchTweetInNode()
{
    cout << "\n\nSearch Tweet in Nodes --\n\n";
    NODE *TEMP = START;

    if (START == nullptr) {
        cout << "null \n\nlist is empty\n";
        return 0;
    }

    cout << "\nEnter User Name To Find Tweets -- ";
    string uname;
    int counter = 0;
    getline(cin, uname);

    while (TEMP != nullptr) {
        if (TEMP->UNAME == uname) { // match found
            cout << "\n[" << TEMP->ID << ", "
                << TEMP->UNAME << ", "
                << TEMP->TWEET << "]\n";
            counter++;
        }
        TEMP = TEMP->NEXT;
    }
}

```

```

    if (counter == 0) {
        cout << "\nNo Tweet Found..\n\n";
    }
}

```

3.9 Count Nodes — O(1)

Concept: Because countNode is updated on every insert and delete, asking the size is constant-time — no scan required.

Time Complexity: O(1).

Algorithm / Steps:

38. Print the value of the member variable countNode.

Method countNodes()

```

void countNodes()
{
    system("cls");
    cout << "Count Nodes Module --\n\n";
    cout << "Total Number of Nodes -- "
        << countNode << " Nodes Available\n\n";
}

```

3.10 Menu Driver — MainMenu()

All singly-list operations are exposed through a single console menu. getch() reads a key without waiting for Enter, and each branch dispatches to the matching method.

Method MainMenu()

```

int MainMenu()
{
    system("cls");
    cout << "=====  

    cout << "1. Insertion Operation\n";
    cout << "2. Traversal Operation\n";
    cout << "3. Delete Operation\n";
    cout << "4. Search Operation\n";
    cout << "5. Count Nodes\n";
    cout << "6. Go to Home Menu\n";
    cout << "7. Exit\n";
    cout << "Choose One of them -- ";

    char c = getch();
    if (c == '1') insertionOperation();
    else if (c == '2') showNode();
    else if (c == '3') deleteOperation();
    else if (c == '4') searchTweetInNode();
    else if (c == '5') countNodes();
    else if (c == '6') MainLinkList::MainLinkListModule();
    else if (c == '7') { thanksLine(); exit(0); }
}

```

Part 4 — Doubly Linked List: All Operations

Node Structure (NODED)

A doubly-linked node carries TWO pointers — PREV and NEXT — letting us walk the list in either direction. The head's PREV is nullptr and the tail's NEXT is nullptr.

File: doublyll.cpp — class NODED

```
class NODED
{
public:
    NODED *PREV;      // <-- backward pointer
    string ID;
    string UNAME;
    string TWEET;
    NODED *NEXT;      // <-- forward pointer

    NODED(string x, string y, string z)
    {
        PREV = nullptr;
        ID = x;
        UNAME = y;
        TWEET = z;
        NEXT = nullptr;
    }
};
```

Memory layout:

```
NULL <- [TWEET1003 | @niharika] <-> [TWEET1002 | @pooja] <-> [TWEET1001 | @deepak] -> NULL
    ^ START                                ^ END
```

List Class — DLinkedList

File: doublyll.cpp — class DLinkedList

```
class DLinkedList
{
public:
    string TID;
    int TCNT;
    string UName, UTweet;
    NODED *START, *END;
    int countNode;

    DLinkedList(){
        TCNT = 1001;
        countNode = 0;
        START = nullptr;
        END = nullptr;
    }
};
```

4.1 Insert at Beginning — $O(1)$

Concept: Same as singly, but with one extra step: the old head's PREV pointer must now point at the new node.

Time Complexity: $O(1)$.

Algorithm / Steps:

39. Allocate a new NODED.
40. If list is empty, set $START = END = TEMP$.
41. Else: $TEMP \rightarrow NEXT = START$, then $START \rightarrow PREV = TEMP$.
42. Move $START = TEMP$.
43. Increment countNode and TCNT.

From insertNode() — $opr == 1$ (Doubly)

```
// opr == 1 --> Insert at Beginning (Doubly)
getData();
NODED *TEMP = new NODED(TID, UName, UTweet);

if (START == nullptr) {
    END = START = TEMP;
}
else {
    TEMP->NEXT = START;    // new -> old head
    START->PREV = TEMP;    // old head <- new (key DOUBLY step)
    START = TEMP;         // head moves to new node
}
countNode++;
TCNT++;
```

4.2 Insert at End — $O(1)$

Concept: Append at the tail and link the new node's PREV back at the old tail. Three pointer assignments — still $O(1)$.

Time Complexity: $O(1)$.

Algorithm / Steps:

44. Allocate a new NODED.
45. If list is empty, set $START = END = TEMP$.
46. Else: $END \rightarrow NEXT = TEMP$, then $TEMP \rightarrow PREV = END$.
47. Move $END = TEMP$.
48. Increment countNode and TCNT.

From insertNode() — $opr == 2$ (Doubly)

```
// opr == 2 --> Insert at End (Doubly)
getData();
NODED *TEMP = new NODED(TID, UName, UTweet);

if (START == nullptr) {
    END = START = TEMP;
}
else {
```

```

    END->NEXT = TEMP;        // old tail -> new
    TEMP->PREV = END;        // old tail <- new (key DOUBLY step)
    END = TEMP;             // tail moves forward
}
countNode++;
TCNT++;

```

4.3 Insert at Given Position — $O(k)$

Concept: Walk to the predecessor, then perform FOUR pointer rewires (compared to two in the singly version) so that both directions stay consistent.

Time Complexity: $O(k)$.

Algorithm / Steps:

49. Allocate a new NODED.
50. Walk TEMP1 to position pos-1.
51. TEMP->NEXT = TEMP1->NEXT (new -> successor).
52. TEMP1->NEXT = TEMP (predecessor -> new).
53. TEMP->PREV = TEMP1 (new <- predecessor).
54. TEMP->NEXT->PREV = TEMP (successor <- new).

From insertNode() — opr == 3 (Doubly)

```

// opr == 3 --> Insert at Given Position (Doubly)
int pos;
getData();
cout << "Enter Position Between 0 to " << countNode << " -- ";
cin >> pos;

NODED *TEMP = new NODED(TID, UName, UTweet);
NODED *TEMP1 = START;

for (int i = 2; i < pos; i++) { // walk to predecessor
    TEMP1 = TEMP1->NEXT;
}

if (START == nullptr) {
    END = START = TEMP;
}
else {
    TEMP->NEXT = TEMP1->NEXT; // new -> successor
    TEMP1->NEXT = TEMP;      // predecessor -> new
    TEMP->PREV = TEMP1;      // new <- predecessor
    TEMP->NEXT->PREV = TEMP;  // successor <- new (DOUBLY)

    if (TEMP1 == END) {
        END = TEMP1->NEXT; // tail moves if needed
    }
}
countNode++;
TCNT++;

```

4.4 Delete from Beginning — $O(1)$

Concept: Advance START to the next node, then null out the new head's PREV pointer so it correctly marks the start.

Time Complexity: $O(1)$.

Algorithm / Steps:

55. If list is empty, abort.
56. If only one node, clear START and END.
57. Else save TEMP = START, move START = START->NEXT, and set START->PREV = nullptr.
58. Delete TEMP.

From deleteNode() — opr == 1 (Doubly)

```
// opr == 1 --> Delete From Beginning (Doubly)
if (START == nullptr) { cout << "LinkedList is Empty..\n\n"; return 0; }
else if (START->NEXT == nullptr) {
    TEMP = START;
    END = START = nullptr;
}
else {
    TEMP = START;
    START = START->NEXT;
    START->PREV = nullptr;    // new head's prev = NULL (DOUBLY)
}
countNode--;
delete(TEMP);
```

4.5 Delete from End — $O(1)$ ★ Key Doubly Advantage

Concept: Unlike the singly list, we do NOT need to scan to the second-last node. Because each node knows its predecessor via PREV, we can step END backward in one move.

Time Complexity: $O(1)$ — the big win over a singly linked list.

Algorithm / Steps:

59. If list is empty, abort.
60. If only one node, clear START and END.
61. Else save TEMP = END, set END = END->PREV, then END->NEXT = nullptr.
62. Delete TEMP.

From deleteNode() — opr == 2 (Doubly)

```
// opr == 2 --> Delete From End (Doubly) --  $O(1)$ 
if (START == nullptr) { cout << "LinkedList is Empty..\n\n"; return 0; }
else if (START->NEXT == nullptr) {
    TEMP = START;
    END = START = nullptr;
}
else {
    NODED *TEMP = END;
    END = END->PREV;    // jump back via prev pointer
    END->NEXT = nullptr; // new tail has no successor
    delete(TEMP);
}
```

```
countNode--;
```

4.6 Delete from Given Position — $O(k)$

Concept: Walk straight to the target (no separate predecessor needed) and rewire the two neighbour pointers. Special case for the tail.

Time Complexity: $O(k)$ for traversal; the rewire itself is $O(1)$.

Algorithm / Steps:

63. If list has only one node, clear START and END.
64. Else walk TEMP from START to position pos.
65. If TEMP == END, set TEMP->PREV->NEXT = nullptr and update END.
66. Else: TEMP->PREV->NEXT = TEMP->NEXT and TEMP->NEXT->PREV = TEMP->PREV.
67. Delete TEMP and decrement countNode.

From deleteNode() — opr == 3 (Doubly)

```
// opr == 3 --> Delete From Given Position (Doubly)
int pos;
cout << "Enter Position for Delete (1 - " << countNode << ") - ";
cin >> pos;

if (pos == 1 && START == END) {
    TEMP = START;
    START = END = nullptr;
}
else {
    TEMP = START;
    for (int i = 1; i < pos; i++) {    // walk straight to target
        TEMP = TEMP->NEXT;
    }
    if (TEMP == END) {                // deleting tail
        TEMP->PREV->NEXT = nullptr;
        END = TEMP->PREV;
        delete(TEMP);
    }
    else {                            // middle node -- O(1) rewire
        TEMP->PREV->NEXT = TEMP->NEXT;
        TEMP->NEXT->PREV = TEMP->PREV;
        delete(TEMP);
    }
}
countNode--;
```

4.7 Forward & Backward Traversal — $O(n)$ each

Concept: Forward traversal walks from START via NEXT — newest tweet first (Twitter feed view). Backward traversal walks from END via PREV — oldest first ("chronological thread" view). Backward traversal is impossible in a singly linked list.

Time Complexity: $O(n)$ for each direction.

Algorithm / Steps:

68. If empty, exit.

69. FORWARD: TEMP = START; while TEMP != nullptr — print, then TEMP = TEMP->NEXT.
 70. BACKWARD: TEMP = END; while TEMP != nullptr — print, then TEMP = TEMP->PREV.

Method showNode() (Doubly)

```
int showNode()
{
    cout << "\n\nDoubly Link List Traversal --\n\n";
    if (START == nullptr) { cout << "list is empty\n"; return 0; }

    cout << "1. Forward Traversal..\n";
    cout << "2. Backward Traversal\n";
    char c = getch();

    if (c == '1') {                // FORWARD: head -> tail
        cout << "null <- ";
        NODED *TEMP = START;
        while (TEMP != nullptr) {
            cout << "[" << TEMP->ID << ", "
                << TEMP->UNAME << ", "
                << TEMP->TWEET << "]" -> ";
            TEMP = TEMP->NEXT;
        }
        cout << "Null\n";
    }
    else if (c == '2') {          // BACKWARD: tail -> head
        cout << "null <- ";
        NODED *TEMP = END;
        while (TEMP != nullptr) {
            cout << "[" << TEMP->ID << ", "
                << TEMP->UNAME << ", "
                << TEMP->TWEET << "]" -> ";
            TEMP = TEMP->PREV;      // <-- only possible because of PREV
        }
        cout << "Null\n";
    }
}
```

4.8 Search by User Name — O(n)

Concept: Identical idea to the singly version — walk forward and compare each node's UNAME with the search term.

Time Complexity: O(n).

Algorithm / Steps:

71. Read uname.
72. Walk TEMP from START to nullptr.
73. If TEMP->UNAME == uname, print the tweet and count it.
74. Print "No Tweet Found" if counter remains zero.

Method searchTweetInNode() (Doubly)

```
int searchTweetInNode()
```

```
{
    cout << "\n\nSearch Tweet in Nodes --\n\n";
    NODED *TEMP = START;
    if (START == nullptr) { cout << "list is empty\n"; return 0; }

    cout << "Enter User Name To Find Tweets -- ";
    string uname;
    int counter = 0;
    getline(cin, uname);

    while (TEMP != nullptr) {
        if (TEMP->UNAME == uname) {
            cout << "\n[" << TEMP->ID << ", "
                << TEMP->UNAME << ", "
                << TEMP->TWEET << "]\n";
            counter++;
        }
        TEMP = TEMP->NEXT;
    }
    if (counter == 0) cout << "\nNo Tweet Found..\n\n";
}
```

Part 5 — Complete Complexity Comparison

Operation	Singly — Time	Doubly — Time	Why
Insert at Beginning	$O(1)$	$O(1)$	Constant pointer rewires.
Insert at End	$O(1)$	$O(1)$	END pointer makes append direct.
Insert at Position k	$O(k)$	$O(k)$	Walk to predecessor.
Delete from Beginning	$O(1)$	$O(1)$	Move START forward.
Delete from End	$O(n)$	$O(1)$ ★	Singly must scan; doubly steps back via PREV.
Delete from Position	$O(k)$	$O(k)$	Walk to target node.
Forward Traversal	$O(n)$	$O(n)$	Visit every node once.
Backward Traversal	Not possible	$O(n)$	Requires PREV pointer.
Search by user name	$O(n)$	$O(n)$	Linear scan.
Count Nodes	$O(1)$	$O(1)$	Maintained as countNode.

When to Use Which?

Use case	Best choice	Reason
Twitter timeline / news feed	Doubly	Constant-time tail delete + bidirectional traversal.
Browser history (Back / Forward)	Doubly	Both directions needed equally often.
Music player playlist	Doubly	Previous / Next track navigation.
Stack / function call chain	Singly	Only top access — doubly wastes memory.
Hash table chaining	Singly	Append-and-traverse pattern only.
LRU cache	Doubly	Move-to-front needs $O(1)$ deletes from any position.

Part 6 — Conclusion

This project implements a Twitter-style timeline using two flavours of Linked List in C++. Each tweet is one node carrying a unique TID, the user name and the tweet text. The Singly list demonstrates how the Twitter feed naturally maps to head-insertion and forward traversal, while the Doubly list adds bidirectional traversal and $O(1)$ tail deletion — exactly the operations a real timeline needs to power features like "scroll to top", "oldest first", and "remove this tweet now."

All operations are wrapped behind a clean menu-driven interface. The driver class MainLinkedList lets the user pick Singly or Doubly at runtime, and every sub-menu cleanly returns to its parent — making the project both a teaching example and a usable mini-application.

Designed & Coded by

Dilkhush aryandeep rohit and kartik



Dezyne École College
www.dezyneecole.com

Thank You